

12.05.25

In the class:

- Kep is demonstrating and commenting on previous approaches.
- All groups dedicated a lot of space / time to describing the environment in a rather hardcoded way.
- Use the Clingonia small app to build small environments
- ...

Idea: try 'broken' tracks.

—

14.05.25

Putting myself back into the context by just running all the simple environments again (check both 'transition' and 'action' atoms):

Env.	Result
Straight line 1 train	Success
Г-turn 1 train	Success
П-turn 1 train	Success
Snake 1 train	Success
Many turns 1 train (different ends)	Success, Success, Success
Bagel 1 train	Success (two answers, just as it's supposed to be given the shape)
Switch trap 1 train	The train flies across the border and then waits at the destination; The second answer is correct.
Basic diamond 1 train	Success
Only curves 1 train	
Diamond for two 2 trains	Success

Single switch 2 trains	Failure (unsatisfiable)
------------------------	-------------------------

CONCERN: when I run on simple environments, but the ones that already include choice junction, the action is still always 'move_forward'. Perhaps, I just didn't encode any other action.

IDEA: encode choice into 'path': deterministic cells will have only either 'move_forward' or 'wait' options. The move must be included in the middle of 'path' atom, like this:

```
path(east, 1, move_forward, 2, east).  
path(west, 1, move_forward, 2, west).  
path(east, 1, wait, east, 1).  
path(west, 1, wait, west, 1).
```

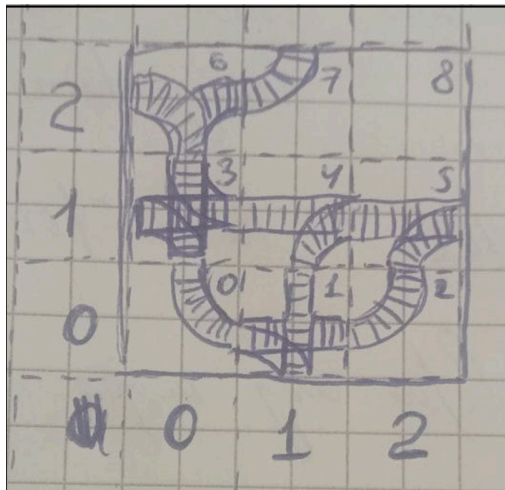
Let's now push to git before we do any experiments with the current code - it's imperfect, but it does work for simple cases. **Done.**

Now I'm free to experiment more. Let's set changing 'path' atoms aside for a while and introduce a few more environments:

- Just many turns.
- Two trains but slightly more complex than a diamond.
- Extreme edge case like a torn route.

—

Let's encode this one: it's 9 cells. Let's borrow from the previous nice cell envs:



"only_curves"

```
train(1).  
start(1, (_, _), _, _).  
end(1, (_, _), _).  
  
cell((0, 0), 72).  
cell((0, 1), 38433).  
cell((0, 2), 2064).  
cell((1, 0), 38505).  
cell((1, 1), 17411).  
cell((1, 2), 17411).  
cell((2, 0), 20994).  
cell((2, 1), 2064).  
cell((2, 2), 0).
```

And fill out types carefully according to the known track types. Let's run it. The expectation is that it will fail, giving 'unsatisfiable'. I also assume that some track types are not yet described.

Forgot to set start and end. Setting them up. Let's try different combinations. Yes, it's unsatisfiable. Why? Most likely the track types are not described.

Yes, they are missing. Let's describe them to create 'path' atoms.

We refer to the table:

https://docs.google.com/spreadsheets/d/1vVDptAVsx0L1_bZe3m-rT9NVDgNmR9R0W8ZQWghZTlw/edit

I've implemented the segment type 38433 now. Let's run the code, but I'm sure there are more undescribed types:

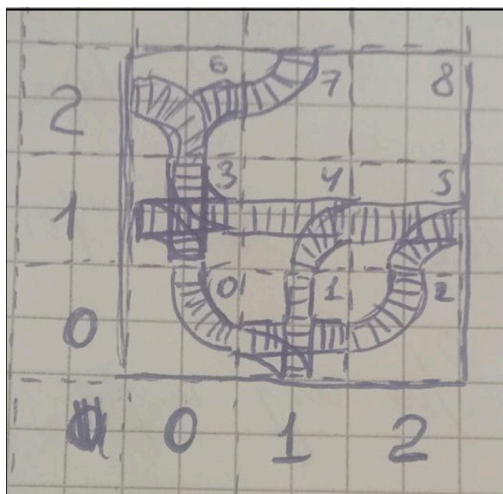
Yes. Let's check: 38505 is also missing. Let's describe using the transitions table. Done. There is still a big likelihood of mistakes, so later I'll need to **re-check all the 'path' atom descriptions**.

Before attempting to run again, let's check if all types included in the new track are described.

17411 (**Done**) and 20994 (**Done!**) are missing! So, I need to describe them first.

Good. Now I can test again. Still I must be ready that the answer will be 'unsatisfiable'.

No, there are as many as **5 answers!** Let's explore them carefully and see where they come from (and if where they come from is legit):



"only_curves"

```
Answer: 1
transition(1,0,6,3)
transition(1,1,3,2)
transition(1,2,2,1)
Answer: 2
transition(1,0,6,3)
transition(1,1,3,0)
transition(1,2,0,1)
Answer: 3
transition(1,0,6,3)
transition(1,1,3,4)
transition(1,2,4,1)
Answer: 4
transition(1,0,6,5)
transition(1,1,5,2)
transition(1,2,2,1)
```

```
Answer: 5
transition(1,0,6,5) transition(1,1,5,4) transition(1,2,4,1)
```

One answer is correct, others imply some impossible transitions. Where do they come from?

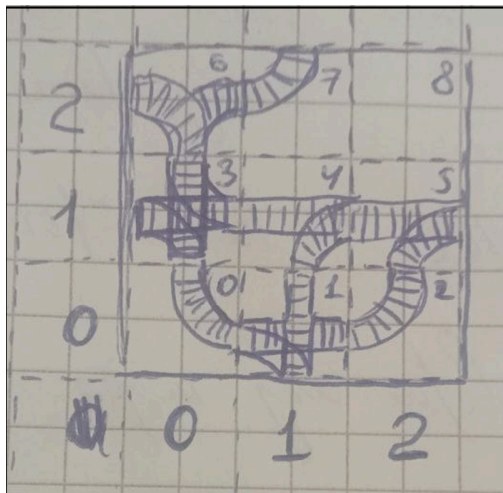
IDEA: transition (1, 0, 6, *move_forward*, 3).

15.05.25

Okay, what's the next thing I can do?

- To check the possibly wrong path descriptions that create impossible transitions.

Let's analyse the impossible transitions one by one:



"only_curves"

Answer: 1

transition(1, 0, 6, 3)

transition(1, 1, 3, 2)

transition(1, 2, 2, 1)

Why does the transition occur between 3 and 2?!

Let's revise the 'path' atoms.

Intuition: it occurs because the condition for a transition to occur is only: there must be a cell for transition, **non-discriminatingly which cell**. This is the problem: the condition only checks if the previous cell is present, not if the previous cell allows for transition. When will it be critical? It will be critical whenever there are two disconnected tracks

in the neighbouring cells - the train will jump from one to another.

Easy fix: replace TYPE != 0 with types allowed for transitions given the starting cell. Let's also first display the 'path' atoms to have a better intuition: are they created where they mustn't be - for example, between 6 and 2?

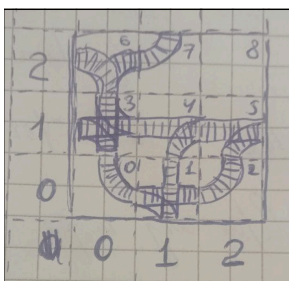
```
path(west,0,north,3) path(south,0,east,1) path(south,0,south,0) path(west,0,west,0)
path(east,2,north,5) path(south,2,west,1) path(south,7,west,6) path(south,2,south,2)
path(south,7,south,7) path(east,2,east,2) path(east,7,east,7) path(north,1,north,4)
path(north,1,east,0) path(west,1,east,0) path(south,3,south,0) path(south,3,east,4)
path(north,3,north,6) path(north,3,west,2) path(west,3,west,2) path(west,3,north,6)
path(east,3,east,4) path(east,3,south,0) path(west,4,west,3) path(west,5,west,4)
path(west,4,south,1) path(west,5,south,2) path(east,4,east,5) path(east,5,east,6)
path(north,4,east,5) path(north,5,east,6) path(north,6,west,5) path(north,6,east,7)
path(east,6,south,3) path(west,6,south,3)
```

There is no such path at all! Let's recheck the description of this complicated crossing again. Let's check the description of the new complicated segment (38505) specifically.

Segment	Correctness
38505	Yes
17411	Yes
20994	Yes

Quick fix: *add prohibited types besides 0.*

Let's look at the impossible transitions again:



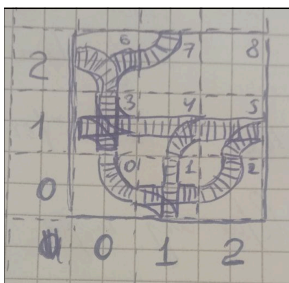
"only_curves"

Answer: 1

`transition(1,0,6,3)` `transition(1,1,3,2)`
`transition(1,2,2,1)`

The transition between cells 3 and 2 is an impossible one. Why does it occur? Let's try to prohibit it by indicating the preceding cell as impossible. But there seems to be no even a PATH atom that would allow this! Isn't it then about how 'transition' atoms are created from 'path' atoms?

Maybe other impossible transitions will give me a clue:



"only_curves"

`transition(1,0,6,3)` `transition(1,1,3,4)`
`transition(1,2,4,1)`

Why does the train go from 4 to 1?

The 'path' for this must be:

`path(east,4,south,1)`

And it **doesn't exist**.

Let's explore other impossible transitions:

`transition(1,0,6,5)` `transition(1,1,5,2)` `transition(1,2,2,1)`

INTUITION: maybe simply a constraint is missing, like: if there is no path
 (path(east, 4, south, 1)), there must be no corresponding transition
 (transition(1, 2, 4, 1)):

Constraint Idea	ASP Phrasing	Code
There must be no transition where there is no path.	There must be no answer set where a transition between two cells does not correspond to a path.	<pre> :- transition(_,_, TO_DIR_1, TO_CELL_1, FROM_DIR_1, FROM_CELL_1), path(TO_DIR_2, TO_CELL_2, FROM_DIR_2, FROM_CELL_2), TO_DIR_1 != TO_DIR_2, TO_CELL_1 != TO_CELL_2, FROM_DIR_1 != FROM_DIR_2, FROM_CELL_1 != FROM_CELL_2.</pre>

But first, let's check it with the right answer: are the correct transitions always supported by paths?

transition(1,0,6,3) transition(1,1,3,0) transition(1,2,0,1)

Let's display path atoms:

```

path(west,0,north,3) path(south,0,east,1) path(south,0,south,0) path(west,0,west,0)
path(east,2,north,5) path(south,2,west,1) path(south,7,west,6) path(south,2,south,2)
path(south,7,south,7) path(east,2,east,2) path(east,7,east,7) path(north,1,north,4)
path(north,1,east,0) path(west,1,east,0) path(south,3,south,0) path(south,3,east,4)
path(north,3,north,6) path(north,3,west,2) path(west,3,west,2) path(west,3,north,6)
path(east,3,east,4) path(east,3,south,0) path(west,4,west,3) path(west,5,west,4)
path(west,4,south,1) path(west,5,south,2) path(east,4,east,5) path(east,5,east,6)
path(north,4,east,5) path(north,5,east,6) path(north,6,west,5) path(north,6,east,7)
path(east,6,south,3) path(west,6,south,3)
```

INTUITION: directions must be taken into account, since otherwise the constraint will be satisfied even with a wrong path.

IDEA: expand 'transition' atoms to include a comprehensive (exhaustive!) description of what's happening with the train, so that we can constrain effectively later.

LLM: no, it's not bad if it serves the purpose.

Let's expand the transition atoms - it will let us constrain more easily.

I've created a new 'motion' file to experiment bolder.

First, let's include 'movement' into 'path'. Let's also test it on the simplest envs.

Let's simply remove more complex tracks for prototyping.

OBSERVATION: expanding path and transition atoms with more variables ruins all the constraints - they start to be based on nonexistent atoms.

Let's test the new model with expanded 'transition' atoms on basic environments:

Environment	Success
Straight line 1 train	Yes
Г-turn 1 train	Yes
П-turn 1 train	Yes
Snake 1 train	Yes
Bagel 1 train	No => Yes
Single Switch 1 train	Yes
Switch Trap 1 train	No => Yes
Many turns 1 train	...
Only curves 1 train	...
Diamond Crossing 1 train	Yes
Diamond Crossing 2 trains	Yes

Why doesn't Bagel work? For a simple reason: the tracks that are used in it aren't described in the new 'path' atom creation. Let's add them: 6672 (**Done**) and 16458 (**Done**).

Later, I'll need to add waiting opportunities to these tracks.

QUESTION: can the same type of atoms be created by two different choice rules?

LLM: yes, but it has implications.

Diamond crossing requires another addition to the path atoms. It seems that there is no choice on the diamond crossing, so all the actions will be either "move_forward" or "wait". **Done**.

For 2 trains in the diamond crossing, I definitely need to add waiting opportunities.

DISCOVERY:

- waiting already works thanks to the straight parts;
- There are suspicious 'transition' atoms - **fixed**.

Let's explore the output more:

```
transition(1,0,3,move_forward,4) transition(1,1,4,move_forward,5)
transition(2,0,1,wait,1) transition(2,1,1,move_forward,4)
transition(2,2,4,move_forward,7) transition(2,3,7,wait,7)
```

It must work even if we shorten the waiting time for the second train:

```
transition(1,0,3,move_forward,4) transition(1,1,4,move_forward,5)
transition(2,0,1,wait,1) transition(2,1,1,move_forward,4)
transition(2,2,4,move_forward,7)
```

Yes, it's enough for the second train to wait for 1 moment to avoid collision.

In the final output it looks very readable:

```
action(1,0,move_forward) action(1,1,move_forward) action(2,0,wait)
action(2,1,move_forward) action(2,2,move_forward)
```

To run a single switch environment, I need to have the 3089 track type described. It seems it has NEVER been described! **Done.**

```
action(1,0,move_forward) action(1,1,move_forward)
action(1,2,turn_left) action(1,3,move_forward)
action(1,4,move_forward) action(1,5,move_forward)
```

16.05.25

This is just one train running on the track. What if I risk extending it to two trains?

For this to work well, I need to prescribe waiting times. However, some of the segments that constitute the track are already with waiting time (let's check this). Yes, all 'horizontal' segments are of type 1025, and it must be enough to allow either of the trains to wait. Let's see.

I get the answers! For the first time on this grid for 2 trains! Let's explore and see if they make sense:

Answer	Commentary
Answer: 1 action(1,0,move_forward) action(1,1,move_forward) action(1,2,move_forward) action(1,3,move_forward)	Looks like both trains arrive at their destinations. But looking at 'transition' atoms themselves could be more informative.

<pre> action(1,4,move_forward) action(1,5,move_forward) action(2,0,wait) action(2,1,move_forward) action(2,2,move_forward) action(2,3,move_forward) action(2,4,wait) action(2,5,move_forward) action(2,6,wait) action(2,7,move_forward) </pre>	
* All the remaining answers differ only by the positions in which train 2 waits.	

Let's analyse 'transition' atoms as well:

Answer	Commentary
<pre> transition(1,0,12,move_forward,13) transition(1,1,13,move_forward,14) transition(1,2,14,move_forward,8) transition(1,3,8,move_forward,2) transition(1,4,2,move_forward,1) transition(1,5,1,move_forward,0) transition(2,0,0,wait,0) transition(2,1,0,move_forward,1) transition(2,2,1,move_forward,2) transition(2,3,2,move_forward,3) transition(2,4,3,wait,3) transition(2,5,3,move_forward,4) transition(2,6,4,wait,4) transition(2,7,4,move_forward,5) </pre>	The trains definitely arrive at their destinations, but the thing with the shape of this track is that neither of the trains really needs to wait.

Let's try to give the second train the minimum time required only and see if the environment handles it.

Yes, it does (just one answer!):

```

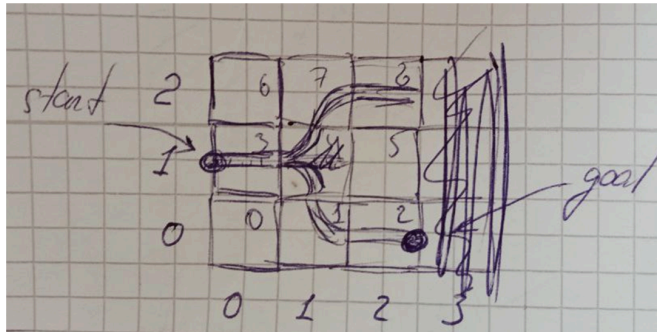
transition(1,0,12,move_forward,13) transition(1,1,13,move_forward,14)
transition(1,2,14,move_forward,8) transition(1,3,8,move_forward,2)
transition(1,4,2,move_forward,1) transition(1,5,1,move_forward,0)
transition(2,0,0,move_forward,1) transition(2,1,1,move_forward,2)
transition(2,2,2,move_forward,3) transition(2,3,3,move_forward,4)
transition(2,4,4,move_forward,5)

```

There is no clash, so the environment is perfectly solved.

“Switch Trap” problem:

The train keeps jumping over the whole field. First thing, let's digitalize the image of the environment for convenient checking.



What makes the train jump? The fact that there is a preceding cell with a track in it. More so - even if I set a prohibitive condition for the types of the track where a transition isn't possible - it would still occur, because these two segments perfectly align. How to solve this?

I must discover the third condition that would allow me to prohibit this movement.

"trap_switch"

BUG: at 4, there must be a turn right, not left! Most likely, it's just a mistake in the encoding of 'path' atoms. Let's check. **Fixed.**

Now the correct answer is completely correct:

```
transition(1,0,3,move_forward,4) transition(1,1,4,turn_right,1)
transition(1,2,1,move_forward,2)
```

INTUITION: use 'grid_size' atom somehow to prohibit jumping; for example, say "if the grid size is 3, prohibit transition between 3 and 2"

INTUITION: create atoms "fringe"

INTUITION: check again if there is indeed 'path' created under the impossible transition;

INTUITION: at ANY moment of time (not only 0 because there might be waiting) there must be no movement out of the starting cell that is not aligned with direction indicated in the start fact.

The latest is the best and the most succinct, it seems. **Doesn't work.**

QUESTION: are trains allowed to wait at the departure station?

Let's go dumb and just implement a hardcoded constraint that will prohibit the wrong transition, later hoping to generalize it.

DISCOVERY: making a very hard (dumb) constraint first, testing and then softening is a good tactic.

Constraint Idea	ASP phrasing	Code
There must be no jumping over the whole grid from edges	There must be no answer set where a horizontal transition happens from the position equal to the height of the grid to the position whose index is smaller by 1.	<pre>:- transition(_, _, FROM_POS, _, TO_POS), grid_size(HEIGHT, WIDTH), FROM_POS = WIDTH, TO_POS = WIDTH - 1.</pre>

It would make sense to introduce a similar constraint for vertical jumps, but *perhaps vertical jumps are not even possible* (this needs to be tested).

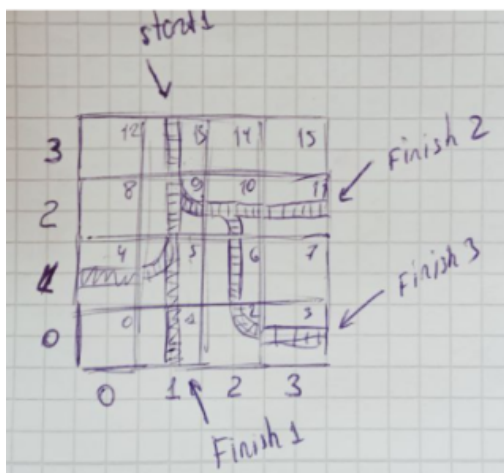
Let's now test if this constraint ruins something in the previous tracks (not all, just sampling!).

Yes, they all work. **Done.**

'Many turns' is currently unsatisfiable perhaps because of lack of encoding of some segments. But first, let's simplify the route of the train on it, picking the straight path only. No, it's already the simplest north-south. Guess: the track in segment 9 is not described. **Done.** What about position 5? Also not! **Done.**

```
transition(1,0,13,move_forward,9) transition(1,1,9,move_forward,5)
transition(1,2,5,move_forward,1)
```

Let's explore if this answer is correct:



"many turns"

Yes, perfect. Let's also have a look at 'action' expression of the same trip:

```
action(1,0,move_forward)
action(1,1,move_forward)
action(1,2,move_forward)
```

Yes, it is correct. Let's now swap end points a little to introduce more complexity. Fails. Guess: track is not described. Index 5633. **Fixed.** The answer:

```

action(1,0,move_forward) action(1,1,turn_right)
action(1,2,move_forward)

```

Testing the third end (expect no problems since all tracks are 'old'):

```

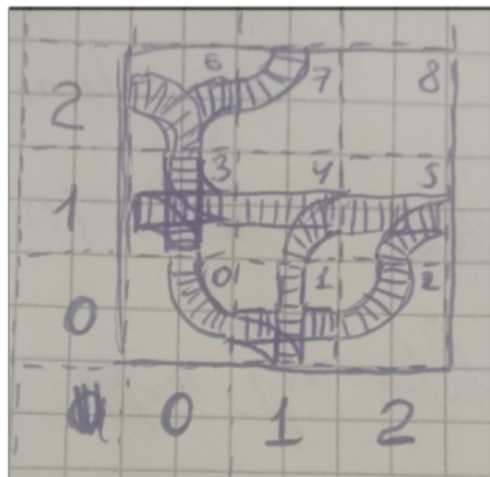
action(1,0,move_forward) action(1,1,turn_right)
action(1,2,turn_right) action(1,3,move_forward)
action(1,4,move_forward)

```

Perfectly solved for 1 train!

Let's test on the most convoluted one - "only curves" - and then proceed to more 2 train environments.

Fails. Guess: tracks are not described. Let's check one by one. Yes: 20994. Fixing. Fixed. Testing: fails. Checking more tracks: copying from movement_1! **Fixed**. Answer:



"only_curves"

Answer: 1

```

action(1,0,move_forward)
action(1,1,turn_right)
action(1,2,move_forward)

```

Answer: 2

```

action(1,0,move_forward)
action(1,1,move_forward)
action(1,2,move_forward)

```

Looks weird. Let's explore 'transition' atoms:

Answer: 1

```

transition(1,0,6,move_forward,3)
transition(1,1,3,turn_right,0)
transition(1,2,0,move_forward,1)

```

Answer: 2

```

transition(1,0,6,move_forward,3) transition(1,1,3,move_forward,0)
transition(1,2,0,move_forward,1)

```

The transition between 3 and 0 is most likely due to a mistake in how track 38505 is described. Checking: weird! There's no path - why is there a transition?

Checking 'path' atoms:

```

path(west,0,move_forward,north,3) path(south,0,move_forward,east,1) path(south,0,wait,south,0) path(west,0,wait,west,0)
path(east,2,move_forward,north,5) path(south,2,move_forward,west,1) path(south,7,move_forward,west,6) path(south,2,wait,south,2)
path(south,7,wait,south,7) path(east,2,wait,east,2) path(east,7,wait,east,7) path(north,6,turn_left,west,5)
path(north,6,turn_right,east,7) path(east,6,move_forward,east,3) path(west,6,move_forward,east,3)
path(south,3,move_forward,south,0) path(south,3,turn_right,east,4) path(north,3,move_forward,north,6)
path(north,3,turn_left,west,2) path(west,3,turn_right,north,6) path(west,3,move_forward,west,2) path(east,3,turn_right,south,0)
path(east,3,move_forward,east,4)

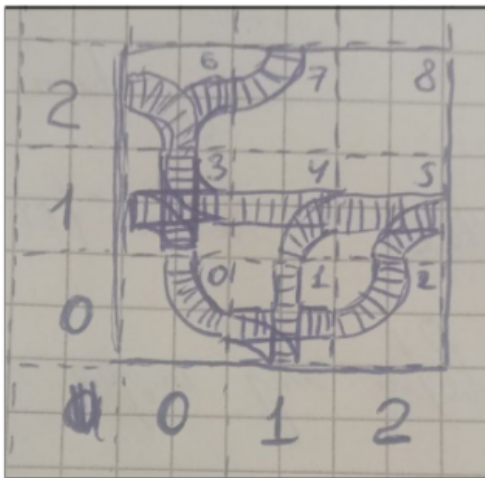
```

IDEA: rename “transition” into “tracker”. **No.** Why? Because semantically “tracker” is not something that you can ‘prohibit’ or control, while ‘transition’ allows for this.

DISCOVERY: the problem of impossible transitions in the answer set is because directions were not accounted for in the generator. To fix this (according to LLM), I need to include the possible directions (only the incoming one at least, but maybe both) into the generator’s rule. **Done.** Now let’s adapt the existing constraints accordingly - otherwise they won’t apply due to a change in the number of attributes that a transition atom holds. **Done.**

OBSERVATION: GPT is very good for finding syntax errors in ASP.

Let’s explore the answers with these new transition atoms:



"only_curves"

Answer: 1

```
transition(1,0,6,east,move_forward,3)
transition(1,1,3,south,move_forward,0)
transition(1,2,0,south,move_forward,1)
```

INTUITION: outgoing directions would also make sense, like this:

```
transition(1,0,6,east,move_forward,3,south)
transition(1,1,3,south,move_forward,0,south)
transition(1,2,0,south,move_forward,1,east)
```

Answer: 2

```
transition(1,0,6,east,move_forward,3)
transition(1,1,3,east,turn_right,0)
transition(1,2,0,south,move_forward,1)
```

Two problems:

- Disconnected transitions (could be solved with adding outgoing transitions to the atom.
- Starting in the wrong direction (probably the constraint on the start direction doesn’t work at all).

Let’s expand the ‘transition’ atom with outgoing directions.

Some constraints don’t work. Let’s re-check. Many answers, let’s analyse to understand what’s missing:

Answer: 1

```

transition(1,0,7,east,wait,7,east)
transition(1,1,7,south,move_forward,6,west)
transition(1,2,6,north,turn_left,5,west) Wrong start, wrong end!
Let's try to prohibit inconsequential movement. Example:

```

```

transition(1,1,3,east,turn_right,0,south) transition(1,2,0,west,wait,0,west)

```

Constraint Idea	ASP phrasing	Code
All transition atoms must be chained through certain pairs arguments.	There must be no answer set where there is a 'transition' atom that doesn't have a pair transition atom paired with it by cell and direction.	<pre> :- transition(TRAIN, MOMENT, _, _, OUT_POS, OUT_DIR), transition(TRAIN, MOMENT + 1, IN_POS, IN_DIR, _, _), OUT_POS != IN_POS, OUT_DIR != IN_DIR. </pre>

It looks like I'm going to replace this constraint:

```

:- transition(TRAIN, MOMENT, _, _, _, TO_CELL, _),
   transition(TRAIN, MOMENT+1, FROM_CELL, _, _, _, _),
   TO_CELL != FROM_CELL.

```

So, it makes sense to remove it instead of adding a new one. This seems to work, but the train now doesn't start and end correctly. Doesn't work and I don't see why. I have plenty of answers again. It looks like the train starts at any position possible where there is a track. Ends as well. Why?

Isn't it easier to just rewrite all constraints from scratch?

Copying the old working constraints table:

Constraint Idea	ASP phrasing	Code
Transitions must not repeat	There must be no answer set where there are two 'transition' atoms with different moments of time but the same transition for the same train (would prohibit circular movement?).	...
Train must not jump between different cells	There must be no answer set where there are two transition atoms where the moment of time	<pre> :- transition(TRAIN, MOMENT, _, _, _, TO_POS, _), </pre>

	is +1 but the transition is nonconsecutive (let's phrase this more precisely: <pre>transition(1,1,2,south,move_forward,1,west) transition(1,2,1,west,move_forward,3,north)</pre> So, there must be no two transition atoms where the train is the same, the moment of time is + 1 but last variable of the first and the pre-last variable of the second are different)	<pre>transition(TRAIN, MOMENT+1, FROM_POS, _, _, _, _), FROM_POS != TO_POS. :- transition(TRAIN, MOMENT, _, _, _, _, TO_DIR), transition(TRAIN, MOMENT+1, _, IN_DIR, _, _, _), TO_DIR != IN_DIR.</pre>
The train must not go back	There must be no two 'transition' atoms where the second one holds the moment of time + 1, but <pre>transition(1,1,0,3) transition(1,2,3,0)</pre>	<pre>:- transition(TRAIN, MOMENT, FROM_POS, IN_DIR, ACTION, TO_POS, OUT_DIR), transition(TRAIN, MOMENT+1, TO_POS, OUT_DIR, ACTION, OUT_POS, IN_DIR).</pre>
The train must arrive at the destination Result: works perfectly on deterministic environments; interesting to explore on 1 switch env.	The 3rd and 4th variables are mutually swapped.	
The train must start from the departure station	There must be no answer set where for the moment 1 (better say: at the <i>indicated departure time</i>!) there is no 'transition' atom that holds 0 as the TO_CELL value.	
The train must arrive at the final station.	There must be no answer set where at the (pre-) last moment of time there is no transition to the respective destination station by the given train.	

Let's debug on the simplest track - straight line. The constraint on connectedness just doesn't work. Why?

DISCOVERY: setting up separate constraints per each variable gave me what I wanted - why?

CONCLUSION: use as targeted, small-scale constraints as possible - better per 1 variable at most.

Which of my constraints now are too strict? (consulting with an LLM)

Let's switch all constraints except the first one off and see if we have the right output among our answers:

```
transition(1,0,0,north,move_forward,3,north)
transition(1,1,3,north,move_forward,6,north)
```

Yes, here's the only correct track.

Let's try switching different constraints on and off to see which one kills it. Good, now we have shrunk down the answer set to just 4, but we still have the correct one:

```
Answer: 1
transition(1,0,0,north,wait,0,north) transition(1,1,0,north,move_forward,3,north)
Answer: 2
transition(1,0,0,north,wait,0,north) transition(1,1,0,north,wait,0,north)
Answer: 3
transition(1,0,0,north,move_forward,3,north) transition(1,1,3,north,move_forward,6,north)
Answer: 4
transition(1,0,0,north,move_forward,3,north) transition(1,1,3,north,wait,3,north)
```

Solved.

Some constraints still might be missing, but let's test on more instances.

DISCOVERY: strict direction at the start solves the cross-field jumping problem for (some) grids.

In the remaining time: try to find out why it doesn't work for 2 trains. Let's also start by running simple environments again and observing where it might falter:

straight_line + , Γ-turn + , Π-turn + , snake + , bagel + , many_turns + , switch_trap + (no more jumping here!), only_curves + + ,

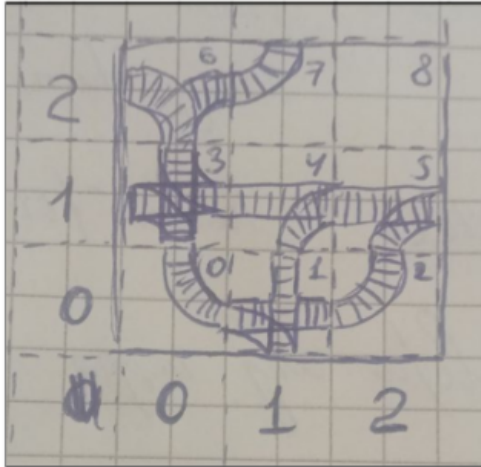
Let's find out why 'many_turns' doesn't work.

H1: not enough time given. **No.**

Truth: wrong direction set at the start.

I think it worked the first time for “only_curves”; let’s explore the answer carefully against the image of the net:

```
transition(1,0,6,east,move_forward,3,east)
transition(1,1,3,east,turn_right,0,south)
```



"only_curves"

```
transition(1,2,0,south,move_forward,1,east)
```

H1: a mistake in ‘path’ atoms; **Yes:** 20994 has a mistake in moving from east or west; **Fixed.**
Let’s rerun:

```
transition(1,0,6,east,move_forward,3,south)
transition(1,1,3,south,move_forward,0,south)
transition(1,2,0,south,move_forward,1,east)
```

Fixed. Let’s get another start / end on only_curves: **start at 5 and end at 7.**

H1: tracks aren’t described. **Yes:** 17411 is not described. **Fixed.** There still might be more undescribed tracks, but let’s try to run: fails. Why? Checking other tracks: 38433 is not described (but neither is it involved in this track). Anyway, let’s implement: **fixed.**

```
transition(1,0,5,west,move_forward,4,west)
transition(1,1,4,west,move_forward,3,west)
transition(1,2,3,west,turn_right,6,north)
transition(1,3,6,north,turn_right,7,east)
```

Let’s explore the answer set. Correct, but in ‘action’ atoms this track is easier to read:

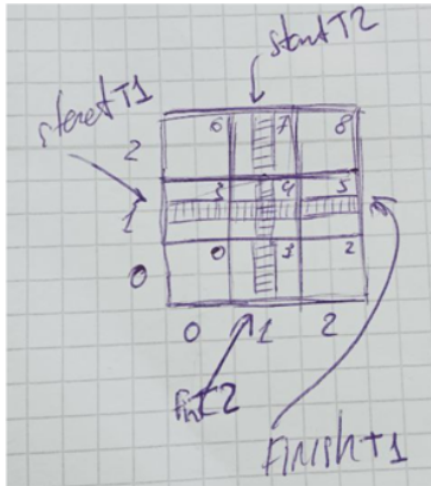
```
action(1,0,move_forward) action(1,1,move_forward)
action(1,2,turn_right) action(1,3,turn_right)
```

Let’s test environments for two trains: fails on the simplest (diamond for two). Why?

17.05.25

Now only one task is prevalent: why does the encoding work so well for a single train but fails even for the simplest two train environment?

To find this out, let's take the simplest 2 train environment - "diamond for two":



"diamond_for_two"

Let's first run the first train on it and the second train on it - separately:

T1 - works:

```
action(1,0,move_forward)
action(1,1,move_forward)
```

```
transition(1,0,3,east,move_forward,4,east)
transition(1,1,4,east,move_forward,5,east)
```

T2 - fails; obvious reason - wrong starting direction. Fixing: **fixed**; the starting and ending positions were also confused in relation to the image; **fixed**; now works - the output is interesting:

```
Answer: 1
transition(2,0,7,south,wait,7,south) transition(2,1,7,south,move_forward,4,south)
transition(2,2,4,south,move_forward,1,south) transition(2,3,1,south,wait,1,south)
Answer: 2
transition(2,0,7,south,move_forward,4,south)
transition(2,1,4,south,move_forward,1,south) transition(2,2,1,south,wait,1,south)
transition(2,3,1,south,wait,1,south)
Answer: 3
transition(2,0,7,south,wait,7,south) transition(2,1,7,south,wait,7,south)
transition(2,2,7,south,move_forward,4,south)
transition(2,3,4,south,move_forward,1,south)
```

Or

```
Answer: 1
action(2,0,wait) action(2,1,move_forward) action(2,2,move_forward) action(2,3,wait)
Answer: 2
action(2,0,move_forward) action(2,1,move_forward) action(2,2,wait) action(2,3,wait)
Answer: 3
action(2,0,wait) action(2,1,wait) action(2,2,move_forward) action(2,3,move_forward)
```

We have this multitude of answers only because the time set for the second train is with a surplus. This must also allow for the passage of two trains (yet I must remember that there is no

constraint against clashing yet). Let's test for two trains without touching the time surplus for the second train.

It works - which means there is no "bad" mistake in the encoding; the answer is:

Answer: 1

```
action(1,0,move_forward) action(1,1,move_forward)  
action(2,0,move_forward) action(2,1,move_forward) action(2,2,wait)  
action(2,3,wait)
```

Answer: 2

```
action(1,0,move_forward) action(1,1,move_forward) action(2,0,wait)  
action(2,1,move_forward) action(2,2,move_forward) action(2,3,wait)
```

Answer: 3

```
action(1,0,move_forward) action(1,1,move_forward) action(2,0,wait)  
action(2,1,wait) action(2,2,move_forward) action(2,3,move_forward)
```

The brown atoms indicate collisions. The answer is still obtainable, but let's now try to prohibit collisions (we can write anew or borrow and modify from the previous version of 'motion' file).

Old Version	New Version
<pre>:- transition(TRAIN_1, MOMENT, _, TO_CELL), transition(TRAIN_2, MOMENT, _, TO_CELL), TRAIN_1 != TRAIN_2.</pre>	<pre>:- transition(TRAIN_1, MOMENT, _, _, _, TO_POS, _), transition(TRAIN_2, MOMENT, _, _, _, TO_POS, _), TRAIN_1 != TRAIN_2.</pre>

Testing: works. Let's analyse the answers:

Answer: 1

```
action(1,0,move_forward) action(1,1,move_forward) action(2,0,wait)  
action(2,1,move_forward) action(2,2,move_forward) action(2,3,wait)
```

Answer: 2

```
action(1,0,move_forward) action(1,1,move_forward) action(2,0,wait)  
action(2,1,wait) action(2,2,move_forward) action(2,3,move_forward)
```

It's all correct: train 1 never waits because it doesn't have any leeway; train 2 has a leeway of 2 and the two answers are the two possible ways to distribute this waiting time. I think there's an opportunity to shrink the waiting time by 1 moment more. Testing: **works**:

Answer: 1

```

action(1,0,move_forward) action(1,1,move_forward) action(2,0,wait)
action(2,1,move_forward) action(2,2,move_forward)

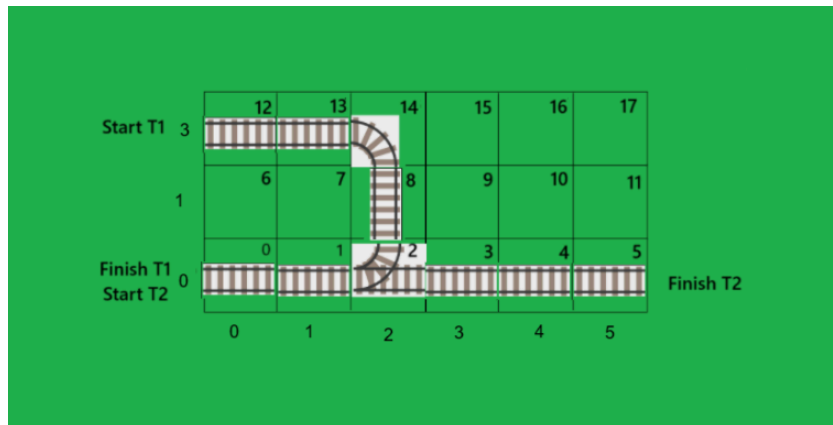
```

There is only one answer now, because there is no leeway in how to locate the waiting time: at the moment 0, train 2 waits to let train 1 pass the crossing).

Let's go to a more complex (only superficially so) environment of a single switch.

Expectation: fails immediately. **Yes.** Why?

H1: a mistake in the environment description;



Again, to debug, let's run trains one by one.

Train 1: fails; so, it's still unsatisfiable for 1 train; this means *there's a basic mistake somewhere here.*

Train 2: the second train runs:

```

action(2,0,move_forward) action(2,1,move_forward)
action(2,2,move_forward) action(2,3,move_forward)
action(2,4,move_forward)

```

H2: some of the tracks that T1 follows are incorrectly turned to "path". **No.**

OBSERVATION: backtracking to shorter paths to debug is a great tactic;

Here's what it let me discover:

Answer: 1

```

transition(1,0,0,east,move_forward,1,east) transition(1,1,1,east,move_forward,2,east)
transition(1,2,2,east,turn_left,8,east)

```

It must not be 'east'! It must be north! Prime suspect: converter of 3089 into path. Checking: yes, the bug was there! **Fixed.** Testing:

```

transition(1,0,0,east,move_forward,1,east)
transition(1,1,1,east,move_forward,2,east)
transition(1,2,2,east,turn_left,8,north)
transition(1,3,8,north,move_forward,14,north)

```

```

transition(1,4,14,north,move_forward,13,west)
transition(1,5,13,west,move_forward,12,west)

```

Perfectly correct. So, now I have a robust encoding, the question is in expanding it to multiple agents / adding speed. Let's try to run two trains:

Again bugs in the 'path' creation. **Fixed.**

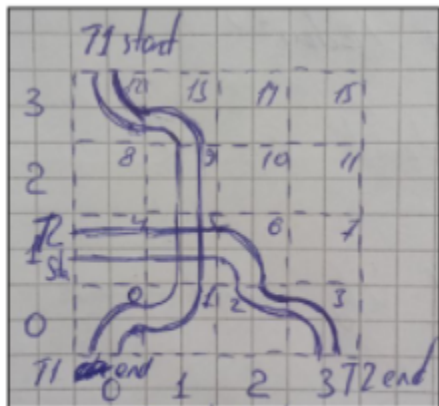
```

Answer: 1
transition(1,0,12,east,move_forward,13,east) transition(1,1,13,east,move_forward,14,east)
transition(1,2,14,east,move_forward,8,south) transition(1,3,8,south,move_forward,2,south)
transition(1,4,2,south,move_forward,1,west) transition(2,0,0,east,move_forward,1,east)
transition(2,1,1,east,move_forward,2,east) transition(2,2,2,east,move_forward,3,east)
transition(2,3,3,east,move_forward,4,east) transition(2,4,4,east,move_forward,5,east)

```

Due to the track geometry, these trains didn't need any waiting time and they traveled successfully. **SUCCESS.**

Task now: introduce more complex environments for 2 trains.



"bowel_for_two"

It's still not much more complex than a simple diamond for two, but let's encode it and test.

```

cell((0,0), 16386) .
cell((0,1), 2064) .
cell((0,2), 72) .
cell((0,3), 4608) .
cell((1,0), 1025) .
cell((1,1), 33825) .
cell((1,2), 4608) .
cell((1,3), 0) .
cell((2,0), 0) .
cell((2,1), 32800) .
cell((2,2), 0) .

```

```

cell((2,3), 0) .
cell((3,0), 72) .
cell((3,1), 4608) .
cell((3,2), 0) .
cell((3,3), 0) .

```

Done. Expectation: trains won't clash even without leeway because of the different distance from the beginning to the intersection.

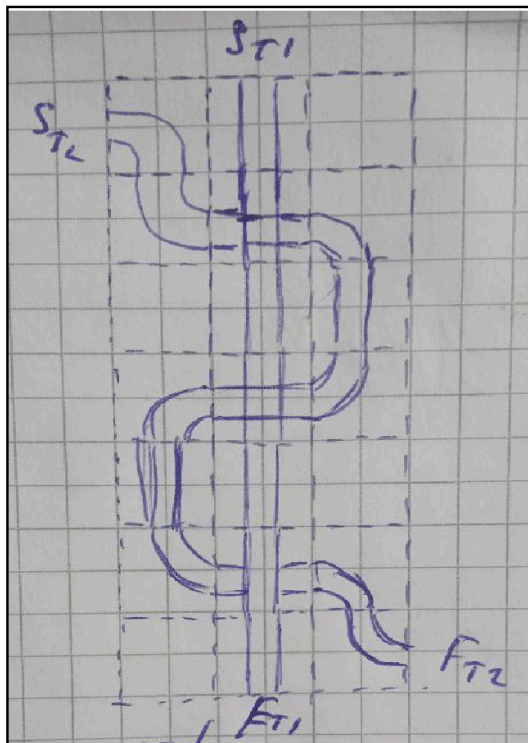
Together: fail; train 1: runs; train 2: fails. Why? Wrong start direction. **Fixed.** Runs. Together: runs; answer:

```
transition(1,0,12,south,move_forward,13,east) transition(1,1,13,east,move_forward,9,south)
transition(1,2,9,south,move_forward,5,south) transition(1,3,5,south,move_forward,1,south)
transition(1,4,1,south,move_forward,0,west) transition(2,0,4,east,move_forward,5,east)
transition(2,1,5,east,move_forward,6,east) transition(2,2,6,east,move_forward,2,south)
transition(2,3,2,south,move_forward,3,east)
```

There's no waiting time needed - because the distance from the beginning is different and the speeds are the same. So, this environment is not really interesting - it's essentially the same as "single switch".

Idea for the next environment: 2 intersections of routes.

Hopefully a little harder, let's try to implement:



"caduceus"

```
cell((0,0), 0).
cell((0,1), 32800).
cell((0,2), 72).

cell((1,0), 72).
cell((1,1), 33825).
cell((1,2), 4608).

cell((2,0), 32800).
cell((2,1), 32800).
cell((2,2), 0).

cell((3,0), 16386).
cell((3,1), 33825).
cell((3,2), 2064).

cell((4,0), 0).
cell((4,1), 32800).
cell((4,2), 32800).

cell((5,0), 72).
cell((5,1), 33825).
cell((5,2), 4608).

cell((6,0), 4608).
cell((6,1), 32800).
cell((6,2), 0).
```

Testing: train 1: **runs**; train 2: fails; analysing: starting direction was set wrong (I constantly confuse east and west!)

Running two: **success**

Answer: 1

```

transition(1,0,19,south,move_forward,16,south) transition(1,1,16,south,move_forward,13,south)
transition(1,2,13,south,move_forward,10,south) transition(1,3,10,south,move_forward,7,south)
transition(1,4,7,south,move_forward,4,south) transition(1,5,4,south,move_forward,1,south)
transition(2,0,18,east,move_forward,15,south) transition(2,1,15,south,move_forward,16,east)
transition(2,2,16,east,move_forward,17,east) transition(2,3,17,east,move_forward,14,south)
transition(2,4,14,south,move_forward,11,south) transition(2,5,11,south,move_forward,10,west)
transition(2,6,10,west,move_forward,9,west) transition(2,7,9,west,move_forward,6,south)
transition(2,8,6,south,move_forward,3,south) transition(2,9,3,south,move_forward,4,east)
transition(2,10,4,east,move_forward,5,east) transition(2,11,5,east,move_forward,2,south)

```

Or:

```

Answer: 1
action(1,0,move_forward) action(1,1,move_forward) action(1,2,move_forward) action(1,3,move_forward)
action(1,4,move_forward) action(1,5,move_forward) action(2,0,move_forward) action(2,1,move_forward)
action(2,2,move_forward) action(2,3,move_forward) action(2,4,move_forward) action(2,5,move_forward)
action(2,6,move_forward) action(2,7,move_forward) action(2,8,move_forward) action(2,9,move_forward)
action(2,10,move_forward) action(2,11,move_forward)

```

Again, this environment doesn't imply any waiting and even any path choosing - everything is deterministic. Yet: **SUCCESS**.

IDEA: would be interesting to try the visualizer at this stage, because it will already look somewhat impressive.

IDEA: the last action atom for any 'standard' answer set can be created like this:

'If there is a transition atom such that there is no transition atom for a later moment, create an 'action' atom with +1 moment for it.'